

Student number: D22127697

Programme code: TU059

Student name: Anais Blenet

Stream: Data science

Course name: Programming with Big Data (PROG9813)

Year of program: 1st year (sem2)

Programming with Big data – CA1

Task: complete a ML competition from Kaggle **only** using python 3 spark APIs

1. Problem definition

I have chosen the playground series competition for the month of March 2024, with the task of predicting steel-plate defects. The playground series is organized by Kaggle, is open to everyone of any level with Kaggle merchandise to win. It mainly serves the purpose of training for other competitions and improving one's knowledge in the machine learning domain.

The task of the competition is as follows: to predict the various defects on steel plates using the synthetic dataset provided and any model of your choice. Submissions are evaluated using area under the ROC curve for each of the defect categories and then averaged to give a final score.

2. Data exploration

The competition provides 3 csv files: train, test and sample_submission. We'll be using train to create and evaluate our models, test to generate the predictions and sample_submission serves as a template to submit probability scores of each target variable. It's important to note that the train & test datasets have been synthetically generated.

I imported both train & test using Pyspark and defining a custom schema. The train file has 35 columns, 7 of which are our target variables and 1 which is our id for each steel plate. There are 19 219 rows, and no missing values. The target variables which dictate what type of faults can be found on a steel plate are Boolean, and the predicting features are of float type except steel type which is also Boolean. A steel plate can have one fault, 2 faults or none. Its interesting to note that "no fault" is not a target variable. The test file only contains the predicting features (27 columns) and the id column. There are 12 814 rows, no missing values.

I compiled a data quality report using the summary method on the predicting columns with a float data type, and manually calculated the 25th and 75th percentile and IQR. I used the count method to look for any missing values in both datasets. I also looked for outliers in those columns by creating a function utilizing IQR at a 1.5 threshold to filter data points outside of the lower and upper bounds. The results are below:

Table 1 - Data quality report for Steel train

summary	x_minimum	x_maximum	y_minimum	y_maximum	pixels_areas	x_perimeter	y_perimeter	sum_luminosity
count	19219	19219	19219	19219	19219	19219	19219	19219
mean	709.8547	753.8576	1849756.040	1846605.345	1683.9876	95.6547	64.1241	191846.6782
stddev	531.5442	499.8366	1903553.851	1896295.138	3730.3199	177.8214	101.0542	442024.6941
min	0.0000	4.0000	6712.0000	6724.0000	6.0000	2.0000	1.0000	250.0000
max	1705.00	1713.00	12987661.00	12987692.00	152655.00	7553.00	903.00	11591414.00
q1	-568.8953	-527.8924	-7885955.71	-7889120.66	-112802.762	-5567.5953	-612.3759	-8501526.3218
q3	1988.605	2035.608	11585467.79	11582331.35	116170.738	5758.905	740.624	8885219.678
iqr	852.500	854.500	6490474.500	6490484.000	76324.500	3775.500	451.00	5795582.00
missing	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
outliers	0.0	0.0	1126.0	1120.0	3742.0	3737.0	3008.0	3837.0

summary	min_lum inosity	max_lumino sity	length_conve yer	steelplate_thick ness	edges_ index	empty_ind ex	square_ind ex	outsideX_in dex	edgesX_in dex
count	19219	19219	19219	19219	19219	19219	19219	19219	19219
mean	84.8084	128.6474	1459.3507	76.2131	0.3529	0.4093	0.5745	0.0306	0.6147
stddev	28.8003	14.1970	145.5687	53.9320	0.3190	0.1241	0.2594	0.0473	0.2224
min	0.0000	39.0000	1227.0000	40.0000	0.0000	0.0000	0.0083	0.0015	0.0144
max	196.000	253.000	1794.000	300.000	0.9952	0.9275	1.0000	0.6651	1.0000
q1	-62.1916	-31.8526	1034.1007	-118.7869	-0.3935	-0.2863	-0.1693	-0.4671	-0.1245
q3	231.808	289.1474	1884.6007	271.2131	1.0993	1.1049	1.3183	0.5283	1.3539
iqr	98.0000	107.0000	283.5000	130.0000	0.4976	0.4638	0.4959	0.3318	0.4928
missing	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
outliers	135.0	1555.0	0.0	2173.0	0.0	67.0	0.0	3654.0	0.0
summary	edgesY_ind ex	outsidglob al_index	logofareas	logX_index	logY_inde x	orientation_ind ex	luminosity_ index	sigmoidofareas	
count	19219	19219	19219	19219	19219	19219	19219	19219	
mean	0.8317	0.5919	2.4735	1.3127	1.3897	0.1027	-0.1384	0.5719	
stddev	0.2210	0.4820	0.7606	0.4678	0.4055	0.4877	0.1203	0.3322	
min	0.1050	0.0000	0.7782	0.3010	0.0000	-0.9884	-0.8850	0.1190	
max	1.0000	1.0000	4.5543	2.9973	4.0333	0.9917	0.6421	1.0000	
q1	0.1604	-0.1581	-0.3586	-0.7096	-1.6352	-1.3823	-1.2837	-0.0888	
q3	1.5029	1.3419	5.3055	3.3349	4.4147	1.5878	1.0069	1.2327	
iqr	0.4475	0.5000	1.8881	1.3482	2.0167	0.9901	0.7636	0.4405	
missing	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
outliers	0.0	0.0	581.0	3354.0	14.0	0.0	999.0	0.0	

Table 2 - Data quality report for Steel test

summary	x_minimum	x_maximum	y_minimum	y_maximum	pixels_areas	x_perimeter	y_perimeter	sum_luminosity	
count	12814	12814	12814	12814	12814	12814	12814	12814	
mean	709.3314	753.2749	1846533.46	1842029.4942	1672.9082	96.4595	64.7306	192689.2991	
stddev	531.8785	500.7436	1916427.39	1911425.6111	3725.5190	219.9285	188.5064	481287.1470	
min	0.0000	4.0000	6712.0000	6724.0000	4.0000	2.0000	2.0000	535.0000	
max	1688.0000	1712.0000	12987661.0	12987692.0000	152655.0000	10449.0000	18152.000	11591414.0000	
q1	-556.6686	-527.7251	-7889178.3	-7893696.5058	-112815.341	-7738.7905	-13547.769	-8500469.9509	
q3	1975.3314	2034.2749	11582245.2	11577755.4942	116161.1582	7931.7095	13677.231	8885848.5491	
iqr	844.0000	854.0000	6490474.50	6490484.0000	76325.5000	5223.5000	9075.0000	5795439.5000	
missing	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
outliers	0.0	0.0	767.0	765.0	2485.0	2462.0	2095.0	2547.0	
summary	min_luminosit y	max_lumino sity	length_conv eyer	steelplate_th ickness	edges_ index	empty_ind ex	square_ind ex	outsideX_in dex	edgesX_in dex
count	12814	12814	12814	12814	12814	12814	12814	12814	12814
mean	84.5481	128.6889	1460.1356	75.8687	0.3540	0.4085	0.5736	0.0307	0.6186
stddev	28.7116	14.2810	146.1109	53.4873	0.3190	0.1245	0.2583	0.0487	0.2204
min	0.0000	37.0000	1227.0000	40.0000	0.0000	0.0000	0.0090	0.0015	0.0144
max	203.000	253.000	1727.000	300.000	0.995	0.944	1.000	0.747	1.000
q1	-67.7019	-33.3111	1085.1356	-119.1313	-0.392	-0.2994	-0.1696	-0.5286	-0.1206
q3	236.7981	290.6889	1835.1356	270.8687	1.1004	1.1164	1.3169	0.5900	1.3578
iqr	101.5000	108.0000	250.0000	130.0000	0.4976	0.4720	0.4955	0.3729	0.4928
missing	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
outliers	131.0	853.0	0.0	1427.0	0.0	60.0	0.0	2405.0	0.0
summary	edgesY_inde x	outsidglobal_index	logofareas	logX_ind ex	logY_inde x	orientation_inde x	luminosity_inde x	sigmoidofarea s	
count	12814	12814	12814	12814	12814	12814	12814	12814	
mean	0.8338	0.5850	2.4727	1.3117	1.3881	0.1012	-0.1395	0.5724	
stddev	0.2203	0.4833	0.7611	0.4676	0.4017	0.4881	0.1179	0.3320	
min	0.0667	0.0000	0.7782	0.3444	0.3010	-0.9910	-0.9989	0.1190	
max	1.0000	1.0000	4.6802	2.9385	4.2587	0.9739	0.5917	1.0000	
q1	0.1338	-0.1650	-0.4538	-0.6339	-1.5802	-1.3725	-1.3325	-0.0883	
q3	1.5338	1.3350	5.3992	3.2573	4.3564	1.5748	1.0534	1.2332	
iqr	0.4667	0.5000	1.9510	1.2971	1.9789	0.9825	0.7953	0.4405	
missing	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
outliers	28.0	0.0	586.0	2238.0	16.0	0.0	635.0	0.0	

Both datasets have no missing values and consistent row counts. The mean and standard deviation values are very different for majority of the features which suggests the data distribution for those features are skewed and can contain outliers. That is further supported by the number of outliers found outside of the upper and lower percentile bounds. The most features with the most significant outliers are sum_of_luminosity, pixels_areas, x_perimeter and logX_index, each column's outliers accounting for at least 17% of the data points in that column, so rather

significant. In addition, the majority of the q1 values are negative which further adds to the skewness of the data points. To remedy this, the data will be scaled using a standard scaler before building the models.

Train dataset class balance:

Next, I wanted to look at the distribution of faults across the train dataset. I created a function called `find_target_column` using a for loop and nested if statements to find & count the “true” instances of each class label and the total “false” instances. The results are below. We can see that only 21 steel plates (rows) have more than one type of fault, the most being 2. This constitutes 0.11% of the total training sample. The classes “Stains” and “Dirtiness” also have very low counts, 2.96% and 2.52% respectively. We can clearly see a class imbalance, especially regarding classes with counts lower than 10% of the total pool which is 5 out of 7.

target_col	count	Class	Count	Class %
Other_faults	6540	Other_faults	6540	34.03%
Bumps	4761	Bumps	4761	24.77%
K_scratch	3411	K_scratch	3411	17.75%
Pastry	1465	Pastry	1465	7.62%
Z_scratch	1150	Z_scratch	1150	5.98%
No Fault	818	No_fault	818	4.26%
Stains	568	Stains	568	2.96%
Dirtiness	485	Dirtiness	485	2.52%
K_scratch, Other_faults	18	K_scratch & Other_faults	18	0.09%
K_scratch, Bumps	2	K_scratch & Bumps	2	0.01%
Pastry, K_scratch	1	Pastry & K_scratch	1	0.01%
Total		Total	19219	100%

Table 3 - Class counts for training dataset

This class imbalance needs to be taken into account when choosing a model robust to this issue or applying techniques to handle this imbalance.

Data cleaning:

Rows with multiple classes will be removed as they are statistically insignificant. I used the same function in the previous section to find out the steel plate id which have more than 1 class label, and then used the filter method to remove the associated rows. Since we have removed the multi label portion of the data, we can modify the dataframe to have one column that contains the type of fault. This will help us during the prediction process.

In addition, rows with no faults will also be removed using the filter method as they are statistically insignificant and are not required for the competition evaluation. Dirtiness and Stains classes are also not statistically significant; however, they are required for the competition evaluation therefore I cannot remove them.

I separated the train dataset into 80-20 using a random seed = 42 and saved the new train and test datasets as `train_clean.csv` and `test_clean.csv`. I recalculated the class counts for both new datasets, and we can see that class imbalance is still present.

New train dataset			New test dataset		
Class	Count	Class %	Class	Count	Class %
Other_faults	5271	35.70%	Other_faults	1269	35.10%
Bumps	3808	25.79%	Bumps	953	26.36%
K_scratch	2708	18.34%	K_scratch	703	19.45%
Pastry	1188	8.05%	Pastry	277	7.66%
Z_scratch	941	6.37%	Z_scratch	209	5.78%
Stains	441	2.99%	Stains	127	3.51%
Dirtiness	408	2.76%	Dirtiness	77	2.13%
Total	14765	100%	Total	3615	100%

Table 4 - New datasets class counts

I will proceed with model creation and try to address this class imbalance with model choice and tuning.

3. Model creation

I have chosen to train 4 supervised learning models¹ using the same approach. I used a vector assembler to combine my predicting features into a vector, and a standard scaler to scale them. I then used a pipeline to combine each model's classifier with the assembler and scaler inside a for loop to create 7 pipelines for each of the target classes. Then, I initialized 2 empty lists to store the models and predictions and simple for loops to fit & transform the models, calculate predictions and evaluation metrics such as AUC, accuracy and F-1 score. Here are the results below:

Metric	Model	Pastry	Z_scratch	K_scratch	Stains	Dirtiness	Bumps	Other_faults	Avg
AUC score	DecTrees	0.78047	0.70913	0.94215	0.64694	0.60554	0.74112	0.53377	0.70845
	Logreg	0.85705	0.91143	0.96321	0.98985	0.84752	0.78062	0.69551	0.86360
	Nns	0.85252	0.92125	0.98541	0.98981	0.84964	0.79334	0.71347	0.87221
	Randfs	0.84929	0.92	0.96495	0.98798	0.857	0.78199	0.71885	0.86858
Accuracy	DecTrees	0.92086	0.95546	0.96349	0.98091	0.97759	0.75436	0.67137	0.88915
	Logreg	0.9231	0.94634	0.96321	0.98285	0.9787	0.75297	0.68271	0.88998
	Nns	0.92365	0.94578	0.96459	0.98230	0.97815	0.75962	0.68105	0.89073
	Randfs	0.92338	0.94993	0.96515	0.98285	0.97898	0.73638	0.60151	0.87688
F1 score	DecTrees	0.88534	0.95099	0.96347	0.98109	0.96908	0.74069	0.65427	0.87785
	Logreg	0.89462	0.93871	0.96310	0.98265	0.9687	0.71188	0.6365	0.87088
	Nns	0.89035	0.94114	0.96459	0.98262	0.96938	0.72975	0.68105	0.87984
	Randfs	0.88659	0.9377	0.96495	0.98258	0.96885	0.62458	0.60151	0.85239

Table 5 - Performance of 4 models on default parameters

I chose these 4 models because I thought them appropriate for the problem. Logistic regression is used for binary classification, which is what we have here; decision trees are robust to overfitting and random forest expand on that; and neural networks incorporate logistic functions and can be tuned & fitted with optimization algorithms.

I chose AUC because it is appropriate for binary classification problems, is robust to class imbalance and doesn't depend on a specific threshold like other metrics; it is also what my results will be evaluated on for this competition. I also decided to calculate accuracy and F1 score to get

¹ Logistic regression, Decision trees, Neural networks & Random forest

more information on how the model performs for each class individually. Each model is operating on default parameters; no feature selection techniques or optimization algorithms have been added to them.

I did not set optional hyperparameters for logistic regression, random forest and decision trees; only for neural network model, I set 3 hidden layers (13-6-2 nodes). The latter model has the highest score across all three metrics, 0.87 for AUC, 0.89 for accuracy and 0.89 for F1 score. Therefore, I will choose the neural network model for further tuning, incorporate feature selection techniques and optimization algorithms with the hopes of improving those metrics.

4. Model tuning & evaluation

I decided to apply cross validator method and play around with its parameters and the model's hyperparameters. Some of my results are below, the rest can be found in an excel file that will be attached with the report:

Method	NN params	Metric	Pastry	Z_scratch	K_scratch	Stains	Dirtiness	Bumps	Other_faults	Avg	Difference	Run time
Cross validator 1	layers 13-6-2	AUC score	0.85429	0.92432	0.98587	0.98966	0.86577	0.80028	0.70584	0.87515	0.00294	9m34s
		Accuracy	0.92559	0.95270	0.96432	0.98368	0.97732	0.76155	0.67580	0.89157	0.00083	
		F1-score	0.89741	0.94857	0.96423	0.98383	0.96935	0.73346	0.64232	0.87702	-0.00282	
Cross validator 1	layers 13-9-6-2	AUC score	0.84074	0.92735	0.98373	0.98863	0.85740	0.79840	0.70785	0.87201	-0.00783	11m23s
		Accuracy	0.92393	0.95270	0.96349	0.98202	0.97842	0.75823	0.67524	0.89058	0.01074	
		F1-score	0.89289	0.94887	0.96353	0.98244	0.97163	0.72754	0.64485	0.87596	-0.00388	
Cross validator + seed (100)	layers 13-6-2	AUC score	0.85469	0.92851	0.98551	0.98941	0.85213	0.79825	0.70304	0.87308	0.00087	20m47s
		Accuracy	0.92476	0.95131	0.96459	0.9823	0.97759	0.75712	0.67441	0.89030	-0.00044	
		F1-score	0.89647	0.9476	0.96459	0.9823	0.96861	0.73269	0.63967	0.87599	-0.00385	

Table 6 - Model tuning

The cross validator 1 has been set as follows:

- MLP estimator using scaled features and layers as specified above
- Estimator parameter map using parameter grid builder with maximum iteration 50-100-200 and step size 0.03-0.1
- Evaluator as multiclassification evaluator with f1 metric

The application of the cross validator 1 yields a positive increase for AUC and accuracy score of the model, but a decrease in the F1 score. This means the cross validator has increased the model's ability to distinguish between the positive and negative classes as well as the proportion of correctly classified classes but decreased the model's ability to correctly identify & capture the positive samples. This could suggest an increase in the model's false negative predictions.

Increasing the number of layers increased the model's average accuracy as expected, but decreased average AUC score and F1 score, probably due to the significant class imbalance. Adding a seed = 100 to the parameter map doubled the runtime of the model and only slightly increased average AUC score and decreased the average accuracy & F1 scores.

I also tuned the parameters of the cross validator to increase/decrease the step rate and the maximum iterations and change the solver. The results were negative, decreasing all the average scores and increasing the runtime.

Overall, the model is able to predict true and false classes quite well. Applying the cross validator has given the model an 89.16% accuracy score, 87.70% F1 score and 87.52% AUC score.

The last step is to use this final model to predict a probability score for each class on the original training dataset, export it to the sample submission file and submit it to the competition.

Prediction calculation & submission:

I used the final model with the cross validator 1 using the original steel train dataset to train the model, and the original steel test dataset which I named steel_pred to calculate the predictions.

Next, I used a first for loop to iterate through each model and extract the predicted probabilities with a spark column method and combined them in a single dataframe using an inner join on the id column. The sample submission file shows that I need to submit only the “true” probabilities for each class, and I currently have dense vectors containing both true and false probabilities.

Thus, I proceeded to convert those column vector types into array types using a for loop to iterate through the names of the vector columns and convert them using the spark vector array method. Then, I created a function to select the true probability in each array which is at index = 1 and used that function inside another loop to select the values I needed into a final dataframe.

Lastly, I made sure to check my final dataframe looked ready for submission, and then wrote it to a csv file using the spark write method.

5. Discussion of work

I used a range of spark methods in this assignment, mostly from the sql functions library to manipulate the dataframes for exploration, cleaning and assembling the final predictions. Col was a very big help all throughout. I also used udf to allow me to create and utilize custom functions, such as the ones to find the true probabilities in the final predictions dataframes and identify the target column based on its value. I only used the rdd method once to be able to apply my custom function to find target column based on its value because it wouldn't work otherwise, even with udf; I'm not sure why.

One aspect of this assignment that I failed to address significantly was the class imbalance in the training data. I considered some methods such as over-sampling/under-sampling, but I was not able to implement them due to the difficulty of the task, and lack of time as well.

I also ran into some issues early on with training the model, I incorrectly assumed that I was supposed to train the model with the original train dataset and test it with the original test dataset, only to find out the test dataset did not have the true labels. I am quite content I realized my incorrect assumption before it was too late.

In addition, tuning the model took a lot of time, as some iterations went well over 20 minutes. In the future, I would like to expand my knowledge on spark to make the model training & tuning process more efficient. This would allow me to for example apply an ensemble method and combine 2 supervised machine learning models on this task to yield better results.

6. Submit your entry

I submitted the zip file which was created when I wrote to file the final predictions. However, Kaggle flagged an error when I uploaded it; I have attached it below in addition to what the csv file looks like. It states that the columns required have not been found. I am currently working on troubleshooting the issue and will send you an update on if I was able to submit it.

Link to competition: [Steel-plate defect prediction](#)

Submission Details

 final_submission.csv.zip
Error · 3m ago

ERROR: Unexpected Column: 'crc' (Line 1, Column 1)
ERROR: Required column 'id' could not be found
ERROR: Required column 'Pastry' could not be found
ERROR: Required column 'Z_Scratch' could not be found
ERROR: Required column 'K_Scratch' could not be found
ERROR: Required column 'Stains' could not be found
ERROR: Required column 'Dirtiness' could not be found
ERROR: Required column 'Bumps' could not be found
ERROR: Required column 'Other_Faults' could not be found

UPLOADED FILES

☒ final_submission.csv.zip (879 KiB)

↓

DESCRIPTION

Cancel

Save

CA1.D22127697 > dataset > final_submission.csv > part-00000-25c88529-c328-4260-a1bc-bb4553ad18c3-c000.csv > data

CATUD	MSCV	meter	20kg	data	CA1.D22127697	atch	K_scratch	Stains	DirtIn
0.000199004939246920034	0.000199004939246920034	0.000199004939246920034	0.000199004939246920034	0.000199004939246920034	0.000199004939246920034	0.000199004939246920034	0.000199004939246920034	0.000199004939246920034	0.000199004939246920034
0.009179372348444075	0.15894964337348938	0.0076689827255904675	0.0076689827255904675	0.0076689827255904675	0.0076689827255904675	0.0076689827255904675	0.0076689827255904675	0.0076689827255904675	0.0076689827255904675
0.01258424948900938	0.014718766324222088	0.012278345413586031	0.012278345413586031	0.012278345413586031	0.012278345413586031	0.012278345413586031	0.012278345413586031	0.012278345413586031	0.012278345413586031
0.04414699599146843	0.0067575592547655106	0.0052761981170015335	0.0052761981170015335	0.0052761981170015335	0.0052761981170015335	0.0052761981170015335	0.0052761981170015335	0.0052761981170015335	0.0052761981170015335
0.08682236820459366	0.0037286412902176E-5	0.008347975090146005	0.008347975090146005	0.008347975090146005	0.008347975090146005	0.008347975090146005	0.008347975090146005	0.008347975090146005	0.008347975090146005
0.009283686056733131	0.1709726792760193E-4	0.0350094735622406	0.0350094735622406	0.0350094735622406	0.0350094735622406	0.0350094735622406	0.0350094735622406	0.0350094735622406	0.0350094735622406
0.22078564763069153	0.837479257024825E-4	0.017397316172719002	0.017397316172719002	0.017397316172719002	0.017397316172719002	0.017397316172719002	0.017397316172719002	0.017397316172719002	0.017397316172719002
0.14805704355239868	0.5337834308156744E-5	0.005271099042147398	0.005271099042147398	0.005271099042147398	0.005271099042147398	0.005271099042147398	0.005271099042147398	0.005271099042147398	0.005271099042147398
0.432051399722695E-4	0.15682377852499485E-4	0.9047041535377502	0.9047041535377502	0.9047041535377502	0.9047041535377502	0.9047041535377502	0.9047041535377502	0.9047041535377502	0.9047041535377502
0.0013615472707897425	0.0013528813142329454	0.005367697216570377	0.005367697216570377	0.005367697216570377	0.005367697216570377	0.005367697216570377	0.005367697216570377	0.005367697216570377	0.005367697216570377
3.886881431955844E-4	0.125137543771416E-4	0.9138120277404785	0.9138120277404785	0.9138120277404785	0.9138120277404785	0.9138120277404785	0.9138120277404785	0.9138120277404785	0.9138120277404785
2.259581524413079E-4	0.08155016601085663	0.04055853188037872	0.04055853188037872	0.04055853188037872	0.04055853188037872	0.04055853188037872	0.04055853188037872	0.04055853188037872	0.04055853188037872
0.2845747768878937	0.0017494580242782831	0.005281889345496893	0.005281889345496893	0.005281889345496893	0.005281889345496893	0.005281889345496893	0.005281889345496893	0.005281889345496893	0.005281889345496893
0.2070196270942688	0.009869543835520744	0.00586498325705528	0.00586498325705528	0.00586498325705528	0.00586498325705528	0.00586498325705528	0.00586498325705528	0.00586498325705528	0.00586498325705528
0.0013939588097855449	0.155865943990648E-4	0.005387859884649515	0.005387859884649515	0.005387859884649515	0.005387859884649515	0.005387859884649515	0.005387859884649515	0.005387859884649515	0.005387859884649515
0.08228490501642227	0.047491151839494705	0.06288046300840378	0.06288046300840378	0.06288046300840378	0.06288046300840378	0.06288046300840378	0.06288046300840378	0.06288046300840378	0.06288046300840378
0.0011490638203531504	0.1302664228671044E-4	0.9109417232467102	0.9109417232467102	0.9109417232467102	0.9109417232467102	0.9109417232467102	0.9109417232467102	0.9109417232467102	0.9109417232467102
3.507040385742486E-4	0.11820907579103570	0.10955114686480105	0.10955114686480105	0.10955114686480105	0.10955114686480105	0.10955114686480105	0.10955114686480105	0.10955114686480105	0.10955114686480105
2.631930401548743E-4	0.06844739615917206	0.005662592127910197	0.005662592127910197	0.005662592127910197	0.005662592127910197	0.005662592127910197	0.005662592127910197	0.005662592127910197	0.005662592127910197
0.0734347403049469	0.0010355800949646235	0.0052681234665215015	0.0052681234665215015	0.0052681234665215015	0.0052681234665215015	0.0052681234665215015	0.0052681234665215015	0.0052681234665215015	0.0052681234665215015
5.520136910490692E-4	0.15051281094878912E-4	0.9107964038848877	0.9107964038848877	0.9107964038848877	0.9107964038848877	0.9107964038848877	0.9107964038848877	0.9107964038848877	0.9107964038848877
3.324688004795462E-4	0.06482143700122833	0.005402050446718931	0.005402050446718931	0.005402050446718931	0.005402050446718931	0.005402050446718931	0.005402050446718931	0.005402050446718931	0.005402050446718931
0.005265762563794851	0.1401536960154772E-4	0.03405985236167908	0.03405985236167908	0.03405985236167908	0.03405985236167908	0.03405985236167908	0.03405985236167908	0.03405985236167908	0.03405985236167908
0.0026019716169685125	0.11694402247667313	0.0054896315559744835	0.0054896315559744835	0.0054896315559744835	0.0054896315559744835	0.0054896315559744835	0.0054896315559744835	0.0054896315559744835	0.0054896315559744835

Code:

1. Data quality report

```
import pyspark
from pyspark.sql.types import *
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("data_exploration") \
    .getOrCreate()

#og df schema
> steeltrain_schema = StructType([StructField("id", IntegerType(), True), ...
> steeltest_schema = StructType([StructField("id", IntegerType(), True), ...

#import dataset
steeltrain_path = "dataset/train.csv"
steeltest_path = "dataset/test.csv"
steel_train = spark.read.csv(steeltrain_path, header=True, schema=steeltrain_schema)
steel_test = spark.read.csv(steeltest_path, header=True, schema=steeltest_schema)

#extract col names
target_cols = steel_train.columns[-7:]
feature_cols = steel_train.columns[1:-7]
print(feature_cols, "\n", target_cols)

## SCHEMA
train_schema = steel_train.printSchema()
test_schema = steel_test.printSchema()

## SUMMARY
exclude_cols = [11,12] #exclude boolean pred features
selected_cols = [col_name for i, col_name in enumerate(feature_cols) if i not in exclude_cols]
train_summary = steel_train.select(selected_cols).describe()
test_summary = steel_test.select(selected_cols).describe()

## MISSING VALUES
from pyspark.sql.functions import col, isnan, when, count

missing_train = steel_train.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c)
                                   for c in steel_train.columns])

missing_test = steel_test.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c)
                                  for c in steel_test.columns])

## OUTLIER DETECTION
#function to calculate outliers
def outliers_iqr(df, col_name, threshold=1.5):
    quantiles = df.approxQuantile(col_name, [0.25, 0.75], 0.01)
    q1 = quantiles[0]
    q3 = quantiles[1]
    iqr = q3 - q1
    lb = q1 - threshold * iqr
    ub = q3 + threshold * iqr
    return df.filter((col(col_name) < lb) | (col(col_name) > ub))

#train data
outliers_train = {}
print("Outliers in train data:")
for col_name in selected_cols:
    outliers_train[col_name] = outliers_iqr(steel_train, col_name)
for col_name, outlier_df in outliers_train.items():
    print(f"Outliers in '{col_name}':")
    outlier_count = outlier_df.count()
    if outlier_count > 0:
        print(f"Number of outliers: {outlier_count}")
    else:
        print("0")

#test data
outliers_test = {}
print("\nOutliers in test data:")
for col_name in selected_cols:
    outliers_test[col_name] = outliers_iqr(steel_test, col_name)
for col_name, outlier_df in outliers_test.items():
    print(f"Outliers in '{col_name}':")
    outlier_count = outlier_df.count()
    if outlier_count > 0:
        print(f"Number of outliers: {outlier_count}")
    else:
        print("0")
```


2. Data cleaning

```
import pyspark
from pyspark.sql.types import *
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("data_cleaning") \
    .getOrCreate()

#og df schema
> steeltrain_schema = StructType([StructField("id", IntegerType(), True), ...
> steeltest_schema = StructType([StructField("id", IntegerType(), True), ...

#import dataset
steeltrain_path = "dataset/train.csv"
steeltest_path = "dataset/test.csv"
steel_train = spark.read.csv(steeltrain_path, header=True, schema=steeltrain_schema)
steel_test = spark.read.csv(steeltest_path, header=True, schema=steeltest_schema)

#extract col names
target_cols = steel_train.columns[-7:]
feature_cols = steel_train.columns[1:-7]
print(feature_cols, "\n", target_cols)

## DROP ROWS WITH MULTIPLE TRUE LABELS

id_values = []
#loop through df to find relevant id
for row in steel_train.rdd.collect():
    count_ones = 0
    for column in target_cols: #iterate over each col in target_cols, check 1 value
        if row[column] == 1:
            count_ones += 1
        if count_ones > 1: #append to id_values if count > 1
            id_values.append(row["id"])
            break

#check correct
print(len(id_values))

#remove rows
steel_train1 = steel_train.filter(~steel_train.id.isin(id_values))
steel_train1.count()

from pyspark.sql.functions import col

## DROP ROWS WITH NO FAULT LABEL

#function to find target column based on value
def find_target_column(row):
    columns_with_ones = [] #store columns with "1" values in current row
    for column in target_cols:
        if row[column] == 1:
            columns_with_ones.append(column)
    if columns_with_ones: #for cols with multiple "1"s on same row
        return ', '.join(columns_with_ones)
    else:
        return 'No_fault'

#convert df to RDD of rows & apply function to each row, add result as new col
rdd = steel_train1.rdd.map(lambda row: Row(**row.asDict()))
rdd = rdd.map(lambda row: row + (find_target_column(row),))

#convert RDD back to df
steel_train2 = spark.createDataFrame(rdd, steel_train1.schema.add("target_col", "string"))

#check no_fault exists in new df
steel_train2.filter(col("target_col") == "No_fault").show(5)

#remove rows & check
steel_train3 = steel_train2.filter(~steel_train2.target_col.isin("No_fault"))
steel_train3.filter(col("target_col") == "No_fault").show(5)

## SPLIT CLEAN DATA & WRITE TO FILE

train_ratio = 0.8
test_ratio = 1 - train_ratio
seed = 42
st_train, st_test = steel_train3.randomSplit([train_ratio, test_ratio], seed=seed)

#write to csv file new train & test data
st_train.write.option("header", True).csv("dataset/train_clean.csv")
st_test.write.option("header", True).csv("dataset/test_clean.csv")
```

3. Model creation

```
import pyspark
from pyspark.sql.types import *
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("model_creation") \
    .getOrCreate()

> steel_schema = StructType([StructField("id", IntegerType(), True), ...
#import dataset
steeltrain_path = "dataset/train_clean.csv"
steelttest_path = "dataset/test_clean.csv"
steel_train = spark.read.csv(steeltrain_path, header=True, schema=steel_schema)
steel_test = spark.read.csv(steelttest_path, header=True, schema=steel_schema)

steel_train.cache
steel_test.cache

#extract col names
target_cols = steel_train.columns[-8:-1]
feature_cols = steel_train.columns[1:-8]
print(feature_cols, "\n", target_cols)

from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.classification import LogisticRegression, DecisionTreeClassifier,
RandomForestClassifier, MultilayerPerceptronClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator,
MulticlassClassificationEvaluator

#defining scaler & assembler
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
scaler = StandardScaler(inputCol="features", outputCol="scaled_features")

pipelines = []
for label_col in target_cols:
    #logistic regression
    lr = LogisticRegression(featuresCol="scaled_features", labelCol=label_col)
    lr_pipeline = Pipeline(stages=[assembler, scaler, lr])
    pipelines.append(lr_pipeline)

    #decision tree
    dt = DecisionTreeClassifier(featuresCol="scaled_features", labelCol=label_col)
    dt_pipeline = Pipeline(stages=[assembler, scaler, dt])
    pipelines.append(dt_pipeline)

    #random forest
    rf = RandomForestClassifier(featuresCol="scaled_features", labelCol=label_col)
    rf_pipeline = Pipeline(stages=[assembler, scaler, rf])
    pipelines.append(rf_pipeline)

    #neural networks
    mlp = MultilayerPerceptronClassifier(featuresCol="scaled_features", labelCol=label_col,
    layers=[len(feature_cols), 13, 6, 2])
    mlp_pipeline = Pipeline(stages=[assembler, scaler, mlp])
    pipelines.append(mlp_pipeline)

#model training
models = []
predictions = []
for pipeline in pipelines:
    model = pipeline.fit(steel_train)
    models.append(model)

#model testing
for model in models:
    prediction = model.transform(steel_test)
    predictions.append(prediction)
```

```

#evaluating models
model_names = ["Logistic Regression", "Decision Tree", "Random Forest", "Neural networks"]
for i, prediction in enumerate(predictions):
    #cycle through model names & target cols for model evaluation
    model_name = model_names[i % len(model_names)]
    target_col = target_cols[i // len(model_names)]

    #define model evaluators
    auc_evaluator = BinaryClassificationEvaluator(labelCol=target_col,
metricName="areaUnderROC")
    f1_evaluator = MulticlassClassificationEvaluator(labelCol=target_col,
predictionCol="prediction", metricName="f1")
    acc_evaluator = MulticlassClassificationEvaluator(labelCol=target_col,
predictionCol="prediction", metricName="accuracy")

    #evaluate
    auc = auc_evaluator.evaluate(prediction)
    acc = acc_evaluator.evaluate(prediction)
    f1 = f1_evaluator.evaluate(prediction)

    #print model, target column, and corresponding metrics
    print(f"Model: {model_name}, Target Column: {target_col}")
    print(f"Area under ROC: {auc}")
    print(f"Accuracy: {acc}")
    print(f"F1 score: {f1}\n")

```

4. Model tuning

```

import pyspark
from pyspark.sql.types import *
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("model_tuning") \
    .getOrCreate()

#updated schema
> steel_schema = StructType([StructField("id", IntegerType(), True), ...

#import dataset
steeltrain_path = "dataset/train_clean.csv"
steeltest_path = "dataset/test_clean.csv"
steel_train = spark.read.csv(steeltrain_path, header=True, schema=steel_schema)
steel_test = spark.read.csv(steeltest_path, header=True, schema=steel_schema)

steel_train.cache()
steel_test.cache()

#extract col names
target_cols = steel_train.columns[-8:-1]
feature_cols = steel_train.columns[1:-8]
print(feature_cols, "\n", target_cols)

from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.classification import MultilayerPerceptronClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator,
MulticlassClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

#defining assembler & scaler
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
scaler = StandardScaler(inputCol="features", outputCol="scaled_features")

```

```

#defining classifier & cross validator
pipelines = []
for label_col in target_cols:
    mlp = MultilayerPerceptronClassifier(featuresCol="scaled_features", labelCol=label_col,
    layers=[len(feature_cols), 13, 6, 2])

    evaluator = MulticlassClassificationEvaluator(labelCol=label_col, metricName="f1")
    param_grid = ParamGridBuilder().addGrid(mlp.maxIter, [50, 100, 200]).addGrid(mlp.stepSize,
    [0.03, 0.1]).build()
    cv = CrossValidator(estimator=mlp, estimatorParamMaps=param_grid, evaluator=evaluator,
    numFolds=3)

    mlp_pipeline = Pipeline(stages=[assembler, scaler, cv])
    pipelines.append(mlp_pipeline)

#model fitting & training
models = []
predictions = []
for pipeline in pipelines:
    model = pipeline.fit(steel_train)
    models.append(model)
    prediction = model.transform(steel_test)
    predictions.append(prediction)

#model evaluation
for i, prediction in enumerate(predictions):
    auc_evaluator = BinaryClassificationEvaluator(labelCol=target_cols[i],
    metricName="areaUnderROC")
    f1_evaluator = MulticlassClassificationEvaluator(labelCol=target_cols[i],
    predictionCol="prediction", metricName="f1")
    acc_evaluator = MulticlassClassificationEvaluator(labelCol=target_cols[i],
    predictionCol="prediction", metricName="accuracy")

    auc = auc_evaluator.evaluate(prediction)
    acc = acc_evaluator.evaluate(prediction)
    f1 = f1_evaluator.evaluate(prediction)

    print(f"Area under ROC for {target_cols[i]}: ", auc)
    print(f"Accuracy for {target_cols[i]}: ", acc)
    print(f"F1-score for {target_cols[i]}: ", f1)

```

5. Final predictions

```

import pyspark
from pyspark.sql.types import *
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("final_pred") \
    .getOrCreate()

> steel_schema = StructType([StructField("id", IntegerType(), True), ...
> steelpred_schema = StructType([StructField("id", IntegerType(), True), ...

#import dataset
steeltrain_path = "dataset/train_clean.csv"
steelpred_path = "dataset/test.csv"
steel_train = spark.read.csv(steeltrain_path, header=True, schema=steel_schema)
steel_pred = spark.read.csv(steelpred_path, header=True, schema=steelpred_schema)

steel_train.cache()
steel_pred.cache()

#extract col names
target_cols = steel_train.columns[-8:-1]
feature_cols = steel_train.columns[1:-8]
print(feature_cols, "\n", target_cols)

```

```

from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.classification import MultilayerPerceptronClassifier
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
from pyspark.ml.functions import vector_to_array
from pyspark.sql.functions import col, udf

## MODEL CREATION
#define scaler & assembler
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
scaler = StandardScaler(inputCol="features", outputCol="scaled_features")

#create pipelines for each target class
pipelines = []
for label_col in target_cols:
    mlp = MultilayerPerceptronClassifier(featuresCol="scaled_features", labelCol=label_col,
    layers=[len(feature_cols), 13, 6, 2])

    evaluator = MulticlassClassificationEvaluator(labelCol=label_col, metricName="f1")
    param_grid = ParamGridBuilder().addGrid(mlp.maxIter, [50, 100, 200]).addGrid(mlp.stepSize,
    [0.03, 0.1]).build()
    cv = CrossValidator(estimator=mlp, estimatorParamMaps=param_grid, evaluator=evaluator,
    numFolds=3)

    mlp_pipeline = Pipeline(stages=[assembler, scaler, cv])
    pipelines.append(mlp_pipeline)

#fit model
models = []
for pipeline in pipelines:
    model = pipeline.fit(steel_train)
    models.append(model)

## EXTRACTING & CONCATENATING PROBABILITIES FOR SUBMISSION

combined_probs = steel_pred.select('id')
#iterate through each model and extract predicted probabilities
for i, model in enumerate(models):
    prediction = model.transform(steel_pred) ##redundant FIX

    #rename probability columns
    prob_col_name = f"probability_{target_cols[i]}"
    prediction = prediction.withColumnRenamed("probability", prob_col_name)

    #join each prediction + new col names to combined_probability df
    combined_probs = combined_probs.join(prediction.select('id', prob_col_name), on='id',
    how='inner')

#extract column vectors
comb_cols = combined_probs.columns[1:]
combined_df1 = combined_probs.select("id")

#loop to convert dense vector into array
for i in comb_cols:
    array_col = vector_to_array(col(i), "float32").alias(i + "_vec")
    combined_df1 = combined_df1.join(
        combined_probs.select("id", array_col),
        on="id",
        how="inner"
    )

#define function to return element index 1 = true probability from arrays
def get_true_prob(array):
    if array is not None and len(array) > 1:
        return array[1]
    else:
        return None
get_udf = udf(get_true_prob)

```

```

#array column names of probabilities
new_cols = combined_df1.columns[1:]

#loop through array column names and target col names to append columns to new df
for col_vec, i in zip(new_cols, target_cols):
    combined_df1 = combined_df1.withColumn(i, get_udf(col(col_vec)))

#select relevant columns for submission in final new df
final_cols = target_cols.copy()
final_cols.append("id")
final = combined_df1.select(final_cols)

#check all is well
final.show(5)

#write to csv file
final.write.option("header",True).csv("dataset/final_submission.csv")

```